

## M002 Extension 1 — Redstone Trail Solver

**Topic:** Redstone Trail Solver (3D Cube Maze) · **Course:** Maze Madness · **Level:** Extension (Advanced) ·

**Time:** about 60 minutes

The agent solves a **3D cube maze** on its own, reading a **redstone trail** all around it to decide each step. Drive it with chat: **l** turns left, **r** turns right, **run** starts the solver, **r1** teleports it back.

### 1 · Read the Chat Commands

Each block binds a **chat word** to an action. Type the word, the agent runs the code.

```
def turn_left():  
    agent.turn(LEFT_TURN)  
player.on_chat("l", turn_left)
```

What does the agent do when you type **l** ?

|  |
|--|
|  |
|  |
|  |

```
def go_back():  
    agent.teleport_to_player()  
player.on_chat("r1", go_back)
```

You type **r1** . Where does the agent go? When is that useful?

|  |
|--|
|  |
|  |
|  |
|  |

Why bind small helpers ( **l** , **r** , **r1** ) *before* you write the big **run** solver?

|  |
|--|
|  |
|  |
|  |
|  |

## 2 · Trace the Solver 🔍

This loop runs when you type `run`. It checks redstone in different directions and reacts.

```
while True:
    agent.move(FORWARD, 1)
    if agent.detect(REDSTONE, DOWN) and agent.detect(REDSTONE, RIGHT):
        agent.move(FORWARD, 1)
        agent.interact(LEFT)
    elif agent.detect(BLOCK, RIGHT) and agent.detect(REDSTONE, LEFT):
        agent.move(FORWARD, 1)
        agent.turn(LEFT_TURN)
        agent.move(FORWARD, 1)
    elif agent.detect(REDSTONE, DOWN):
        agent.move(UP, 1)
    elif agent.detect(REDSTONE, UP):
        agent.move(DOWN, 4)
```

`while True` keeps going. When does it stop on its own?

|  |
|--|
|  |
|  |
|  |

In your own words: what does `detect(REDSTONE, DOWN)` tell the agent?

|  |
|--|
|  |
|  |
|  |
|  |

The maze is 3D. Which two lines move the agent between floors? What does each do?

|  |
|--|
|  |
|  |
|  |
|  |

Why does the order of the `if` / `elif` checks matter? What changes if the `elif` `agent.detect(REDSTONE, DOWN)` check came *first*?

|  |
|--|
|  |
|  |
|  |
|  |

### 3 · Spot the Bug 🐛

Read what each block should do, rewrite it, then explain the fix.

**Bug A** — The solver should keep going until the maze is done, checking the trail every step.

```
agent.move(FORWARD, 1)
if agent.detect(REDSTONE, DOWN):
    agent.move(UP, 1)
```

**Hint:** this checks only once. What wraps a block so it repeats?

**Write the fix:**

|  |
|--|
|  |
|  |
|  |
|  |

**Why was it wrong? Why does your fix work?**

|  |
|--|
|  |
|  |
|  |
|  |

**Bug B** — When there is redstone **down and right**, step forward, then flip a lever on the **left**.

```
if agent.detect(REDSTONE, DOWN) or agent.detect(REDSTONE, RIGHT):
    agent.interact(LEFT)
```

**Hint:** "down **and** right" — both must be true. Move forward before you interact.

**Write the fix:**

|  |
|--|
|  |
|  |
|  |
|  |

**Why was it wrong? Why does your fix work?**

|  |
|--|
|  |
|  |
|  |
|  |

---

## 4 · Write a Branch 📌

Add one more `elif` to the solver. When there is redstone straight **ahead**, the agent should turn **right** and move forward.

**Write the branch:**

|  |
|--|
|  |
|  |
|  |
|  |

**In one line: where in the loop should this go, and why?**

|  |
|--|
|  |
|  |
|  |

---

## 5 · Make It Your Own

Open the extension world and run the solver with `run`. Change **one thing**, predict, then test.

Pick **one** (or your own):

- Add a `back` command that turns the agent around (two right turns).
- Change `agent.move(DOWN, 4)` to a new number and watch where it lands.
- Add an `elif` that reacts to redstone straight **ahead**.

**Which change? Write the code.**

|  |
|--|
|  |
|  |
|  |
|  |
|  |
|  |
|  |
|  |

**Predict:** what will happen?

|  |
|--|
|  |
|  |
|  |

**Test it.** What actually happened? If it surprised you, why?

|  |
|--|
|  |
|  |
|  |
|  |

## 6 · Show Your Work 📷 🎥

When the agent finishes the cube maze, come back.

Record **one video** (a phone is fine). Show two things:

**1 · Your code.** Scroll through it. Say what each part does.

**2 · Your build.** Point the camera. Name the parts.

Fill the blanks:

Today I built \_\_\_\_\_.

I built it using this code: \_\_\_\_\_.

In this code I used \_\_\_\_\_.

The hardest part was \_\_\_\_\_.

That part was hard because \_\_\_\_\_.

**Write your lines here, then say them in your video.**

|  |
|--|
|  |
|  |
|  |
|  |
|  |
|  |
|  |
|  |
|  |
|  |
|  |
|  |
|  |
|  |

**Submit** ✅

Send this worksheet + your video to teacher on KakaoTalk.